Aplicaciones de IA · INACAP · Segunda evaluación

Guía de estudio: detección de phishing con LSTM

Todo lo que necesitan dominar para la defensa, explicado desde cero. Sirve para los dos: el profesor puede preguntarle a cualquiera por el funcionamiento general.

Estudiante 1 · problema y modelo | Estudiante 2 · código y resultados

Defensa: 5 min por persona + preguntas

El hilo completo en un minuto

El enunciado pide coherencia entre las partes. Si entienden esta cadena, pueden responder casi cualquier pregunta.

Problema
Phishing
Correos falsos que imitan a un banco o a TI para robar claves. Basta que caiga una persona.

Modelo
LSTM
Un correo es una secuencia de palabras; la LSTM lee en orden y recuerda.

Implementación
Keras en Colab
Limpiar, pasar a números, entrenar con validación y parada temprana.

Resultado
Parcial
Detecta 94,5% de los fraudes. Se escapan los bien escritos.

DILO ASÍ

"Queremos clasificar cada correo como seguro o fraudulento a partir de su texto. Como el texto es una secuencia, usamos una LSTM, que lee palabra por palabra y guarda memoria. La entrenamos con 11.629 correos etiquetados y en la prueba detectó 1.218 de 1.289 fraudes. Resuelve el problema en parte: los fraudes bien redactados, sin urgencia, todavía se escapan."

Las cifras que tienen que saber de memoria

Dato	Valor	Para qué sirve recordarlo
Correos en el dataset	18.650	11.322 seguros y 7.328 phishing
Después de limpiar	17.103	se quitaron vacíos y duplicados

Entrenamiento / validación / prueba	11.629 / 2.053 / 3.421	prueba 20%, validación 15% del resto
Vocabulario y largo	20.000 / 200	mediana de un correo: 160 palabras
Parámetros	1.313.089	casi todos en el Embedding
Exactitud LSTM	0,967	igual que la regresión logística
Recall de fraude	0,945	la métrica más importante
AUC	0,991	regresión logística: 0,995

Tipos de aprendizaje

Es el criterio 2.1.1 de la evaluación: "reconoce los métodos de aprendizaje de la IA según el origen de la información". Pregunta casi segura.

Tipo	De dónde aprende	Ejemplo del curso
Supervisado	Ejemplos con la respuesta correcta (etiqueta).	Nuestro proyecto: cada correo dice "Safe" o "Phishing". También regresión logística, CNN de dígitos.
No supervisado	Datos sin etiqueta; busca grupos o rarezas.	K-Means (agrupa), Isolation Forest (detecta anomalías).
Por refuerzo	Prueba acciones y recibe premios o castigos.	Un agente que aprende a jugar.

Además, es **clasificación binaria**: la salida es una de dos clases. Si la salida fuera un número continuo (un precio, una temperatura) sería regresión.

IA, Machine Learning y Deep Learning son círculos dentro de círculos. La IA es el campo completo. Machine Learning son los métodos que aprenden de datos. Deep Learning son las redes neuronales con varias capas, que aprenden solas qué rasgos importan. Nuestra LSTM es Deep Learning.

DILO ASÍ

"Es aprendizaje supervisado, porque cada correo del dataset trae su etiqueta. El modelo compara su respuesta con la correcta y corrige sus pesos."

OJO

Si preguntan por Isolation Forest (aparece en el material asociado a fraude): sirve cuando no hay etiquetas y se buscan casos raros. Nosotros sí tenemos etiquetas, así que el aprendizaje supervisado aprovecha más información.

Los datos y sus trampas

E1 E2

Dataset público de Kaggle "Phishing Email Detection". Columnas: el texto del correo y su tipo ("Safe Email" o "Phishing Email"). Está en inglés y mezcla phishing con spam común.

Atajos: cuando el modelo aprende la pista equivocada

Un modelo aprende cualquier patrón que separe las clases, aunque no tenga nada que ver con el fraude. Encontramos tres:

Señal	En seguros	En phishing	Qué hicimos
`enron`	20,3%	0%	Se borra junto con `vince`, `kaminski`, `hou`, `ect`.
`enlaceweb`	30,9%	10,7%	Se deja y se declara como limitación.
Vocabulario académico (`university`)	14,7%	0,9%	Explica que se escape el correo de Soporte TI.

¿Qué es Enron y por qué está en los datos? -

Enron fue una empresa de energía de Estados Unidos que quebró en 2001 por fraude contable. En la investigación se publicaron cerca de medio millón de correos internos. Es uno de los pocos conjuntos grandes de correos reales, así que muchos datasets los usan como correos legítimos.

Si dejamos la palabra, el modelo aprende "si dice enron, es seguro". En la prueba funciona, porque viene del mismo dataset, pero con correos de cualquier otra organización esa regla no sirve. Vince Kaminski era un ejecutivo de Enron con muchísimos correos; `hou/ect` sale de sus direcciones internas (Houston, Enron Capital & Trade).

¿Por qué "enlaceweb" aparece más en correos seguros? -

Esperaríamos lo contrario: el phishing casi siempre trae un enlace. Lo probable es que en muchos correos de phishing del dataset los enlaces vengan cortados con espacios ("http : / / ..."), y nuestra expresión regular no los reconoce. Además, los correos seguros (listas académicas, Enron) traen muchos enlaces normales. Conclusión para la defensa: el modelo no usa el enlace como señal de fraude, y eso es una limitación de los datos.

DILO ASÍ

"Antes de entrenar revisamos los datos. Encontramos que la palabra 'enron' delataba la clase y la quitamos, para que el modelo aprenda del contenido y no del origen del correo."

Del texto a números

E1 E2

Una red neuronal solo hace multiplicaciones y sumas. No entiende letras. El camino es: texto, luego texto limpio, luego una lista de números enteros, luego vectores.

1. **Limpiar** con `limpiar()`: minúsculas, sin HTML, enlaces a `enlaceweb`, solo letras, sin las palabras de Enron.
2. **Vocabulario**: `TextVectorization` cuenta las palabras del entrenamiento y asigna un número a las 20.000 más frecuentes. El 0 queda para relleno y el 1 para palabras desconocidas.
3. **Largo fijo**: cada correo queda en 200 números. Los cortos se rellenan con ceros al final; los largos se cortan y se pierde lo que viene después de la palabra 200.
4. **Embedding**: cada número se reemplaza por un vector de 64 valores.

```
# ejemplo ilustrativo (los números reales dependen del vocabulario)
"verify your account now"  →  [412, 37, 96, 158, 0, 0, 0, ... 0]   # 200 números
```

Lo complejo: ¿qué es de verdad un Embedding? -

Es una tabla de 20.000 filas por 64 columnas. La fila 412 es el vector de la palabra 412. Al comienzo la tabla tiene valores al azar; durante el entrenamiento se ajusta igual que cualquier otro peso.

Como la red necesita separar fraudes de correos seguros, termina poniendo cerca las palabras que cumplen un papel parecido: `verify`, `confirm` y `suspended` quedan con vectores parecidos porque aparecen en el mismo tipo de correo. Nadie le dice qué significa cada palabra; lo deduce de los ejemplos.

¿Por qué no usar directamente el número 412? Porque el número es un nombre, no una cantidad: la palabra 412 no es "el doble" de la 206. Un vector de 64 valores sí puede expresar parecidos.

`mask_zero=True` le avisa a la LSTM que los ceros son relleno y que no los lea.

OJO

`adapt()` se llama solo con los datos de entrenamiento. Si el vocabulario se armara con todo, la prueba "filtraría" información al modelo y los resultados se verían mejores de lo que son (fuga de datos).

La neurona y las funciones de activación

E1

Una neurona artificial recibe valores, multiplica cada uno por un **peso**, suma todo, le agrega un **sesgo** y pasa el resultado por una **función de activación**.

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_{64} \cdot x_{64} + b \quad \rightarrow \quad \text{salida} = \text{activación}(z)$$

- El **peso** dice cuánto importa cada entrada (y si empuja hacia arriba o hacia abajo).
- El **sesgo** corre el punto de partida, como el intercepto de una recta.
- La **activación** agrega curvatura. Sin ella, muchas capas juntas serían una sola operación lineal.

LA SIGMOIDE EN NÚMEROS

z (lo que llega)	sigmoide(z)	Decisión con umbral 0,5
-6	0,2%	SEGURO
-3	4,7%	SEGURO
-1	26,9%	SEGURO

	+0	50,0%	umbral
	+1	73,1%	FRAUDULENTO
	+3	95,3%	FRAUDULENTO
	+6	99,8%	FRAUDULENTO

Valores grandes positivos se acercan a 100%, grandes negativos a 0%. En z = 0 da exactamente 50%.

Función	Rango	Dónde la usamos
Sigmoide $1/(1+e^{-z})$	0 a 1	Salida (probabilidad) y compuertas de la LSTM.
tanh	−1 a 1	Dentro de la LSTM, para el contenido de la memoria.
ReLU max(0, z)	0 a ∞	No la usamos; es la típica de capas ocultas en CNN.

DILO ASÍ

"La sigmoide aplasta cualquier número a un valor entre 0 y 1. Por eso la usamos en la última neurona: el resultado se lee como la probabilidad de que el correo sea fraude."

RNN y LSTM: leer con memoria

E1

Una red normal (densa) recibe todo de una vez y no sabe qué palabra vino primero. Una **red recurrente (RNN)** lee una palabra a la vez y le pasa a la siguiente un resumen de lo leído (estado oculto).

Lo complejo: ¿por qué una RNN simple "olvida"? -

Para aprender, el error se propaga hacia atrás por cada palabra. En una RNN simple, en cada paso hacia atrás el gradiente se multiplica por números pequeños. Después de muchas palabras queda casi en cero: es el **desvanecimiento del gradiente**. La red no logra conectar una palabra del final con una del principio.

La LSTM agrega una **memoria de celda** que se actualiza sumando y borrando de a poco, en vez de reescribirse entera en cada paso. Por esa vía el gradiente viaja más lejos sin apagarse.

Las tres compuertas

Cada compuerta es una sigmoide: entrega valores entre 0 (cerrado) y 1 (abierto) para cada uno de los 64 valores de la memoria.

UNA FRASE LEÍDA PALABRA POR PALABRA

Ilustrativo para entender la idea; no sale de nuestro modelo.

Palabra	Olvido	Entrada	Memoria acumulada
`please`	casi todo pasa	guarda poco: palabra muy común	nada sospechoso todavía
`confirm`	mantiene	guarda bastante: pide una acción	alguien pide hacer algo
`your`	mantiene	casi nada: relleno	sigue el pedido de acción
`password`	mantiene lo importante	guarda mucho: "password" junto a "confirm"	pedido de clave: fuerte señal
`within`	mantiene	guarda poco	pedido de clave + plazo
`hours`	mantiene	completa la idea de plazo	pedido de clave con urgencia

Al terminar, el resumen de 64 valores va a la capa Dense, que entrega la probabilidad.

- **Olvido**: qué borrar de lo que ya sabe.
- **Entrada**: qué guardar de la palabra nueva.
- **Salida**: qué parte de la memoria entregar como resumen.

Al llegar a la última palabra, la LSTM entrega **64 valores** que resumen todo el correo. Esa es la entrada de la capa final.

DILO ASÍ

"La LSTM tiene una memoria y tres compuertas: una decide qué olvidar, otra qué guardar de la palabra nueva y otra qué entregar. Así puede relacionar 'confirm' con 'password' aunque estén separadas por varias palabras."

Nuestra arquitectura, capa por capa

E1 E2

`Input(200)`	200 números enteros por correo	0
`Embedding(20000, 64)`	cada número pasa a un vector de 64	1.280.000
`LSTM(64)`	lee en orden y resume en 64 valores	33.024
`Dropout(0.5)`	al entrenar apaga la mitad al azar	0
`Dense(1, sigmoid)`	una probabilidad entre 0 y 1	65

¿De dónde salen esos números de parámetros? (pregunta de nota alta) -

Embedding: una tabla de 20.000 palabras × 64 valores = **1.280.000**.

LSTM: tiene 4 bloques internos (olvido, entrada, salida y el candidato a memoria). Cada uno mira la palabra actual (64 valores) y el resumen anterior (64 valores), con 64 neuronas y un sesgo cada una:

```
4 × (64 × (64 + 64) + 64) = 4 × 8.256 = 33.024
```

Dense: 64 pesos (uno por valor que entrega la LSTM) + 1 sesgo = **65**.

Dropout no tiene parámetros: solo apaga valores. Total: 1.313.089.

Diferencias con el ejemplo del profesor (noticias falsas)

Aspecto	Profesor	Nosotros	Por qué
LSTM	Dos apiladas (64 y 32)	Una de 64	Más simple y rápida para programar en vivo; el dataset no necesitó más.
Texto a números	Tokenizer + pad_sequences	TextVectorization	Es la herramienta actual de Keras y queda dentro del modelo guardado.

Vocabulario / largo	5.000 / 500	20.000 / 200	Los correos son más cortos (mediana 160) y con vocabulario variado.
Validación	Con los datos de prueba	Conjunto separado	La prueba se usa una sola vez, para no ajustar el modelo a ella.
Parada temprana	No	Sí, paciencia 2	Evita sobreajuste y elige la mejor época.

Cómo aprende la red

E1 E2

Aprender es ajustar 1.313.089 pesos para que el error sea lo más bajo posible. Se repite este ciclo:

1. **Propagación hacia adelante**: un lote de correos pasa por las capas y sale una probabilidad para cada uno.
2. **Cálculo del error** con la función de pérdida.
3. **Retropropagación**: se calcula cuánto aportó cada peso al error (su gradiente), desde la salida hacia atrás.
4. **Descenso del gradiente**: cada peso se mueve un poco en la dirección que baja el error. Lo hace el optimizador Adam.

Función de pérdida: entropía cruzada binaria

```
pérdida = -[ y · ln(p) + (1 - y) · ln(1 - p) ]
```

`y` es la respuesta real (1 fraude, 0 seguro) y `p` la probabilidad que dio el modelo. Si es fraude, solo cuenta −ln(p); si es seguro, solo −ln(1−p).

CUÁNTO CASTIGA CADA ERROR

Real	p dicha	Pérdida	Situación
Fraude (1)	0,99	0,01	acierta seguro
Fraude (1)	0,9	0,11	acierta
Fraude (1)	0,6	0,51	acierta dudando
Fraude (1)	0,4	0,92	se equivoca

Fraude (1)	0,01	4,61	se equivoca con seguridad
Seguro (0)	0,01	0,01	acierta seguro
Seguro (0)	0,9	2,30	se equivoca con seguridad

Con un fraude, decir p = 0,01 cuesta 4,61; decir p = 0,4 cuesta 0,92. Equivocarse *con seguridad* cuesta mucho más, y eso empuja al modelo a no estar seguro de algo falso.

Lo complejo: ¿qué es un gradiente y qué hace Adam? -

Imaginen el error como un terreno con cerros y valles, donde cada punto es una combinación de pesos. El **gradiente** es la pendiente en el punto donde están: dice hacia dónde sube el error y qué tan fuerte. Descenso del gradiente es dar un paso en la dirección contraria, cuesta abajo.

El **tamaño del paso** es la tasa de aprendizaje. Muy grande y salta por encima del valle; muy chica y tarda demasiado. **Adam** ajusta solo el paso de cada peso: recuerda la dirección de los pasos anteriores (como una inercia) y achica el paso de los pesos que cambian mucho. Por eso funciona bien sin tener que afinarlo a mano.

Por qué sigmoide va con entropía cruzada binaria: juntas producen un gradiente simple y proporcional al error (p − y). Si el modelo se equivoca mucho, corrige fuerte; si acierta, corrige poco.

Lote, época y paso

- **Lote (batch) de 32**: los pesos se actualizan cada 32 correos. Con uno solo sería muy ruidoso; con todos, muy lento.
- **Paso**: una actualización. 11.629 ÷ 32 = **364 pasos por época** (el último lote es más chico).
- **Época**: una pasada completa por los 11.629 correos de entrenamiento. Máximo 6.

```
# Paso 10 y 11 del notebook
modelo.compile(optimizer='adam', loss='binary_crossentropy',
               metrics=['accuracy', Recall, Precision])
parada = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
modelo.fit(X_ent_num, y_ent, validation_data=(X_val_num, y_val),
           epochs=6, batch_size=32, callbacks=[parada])
```

Sobreajuste y cómo lo controlamos

E1 E2

Sobreajuste es cuando la red memoriza los correos de entrenamiento en vez de aprender patrones. Se nota porque el error de entrenamiento sigue bajando mientras el de validación se estanca o sube.

Conjunto	Correos	Para qué se usa
Entrenamiento	11.629	Ajustar los pesos y armar el vocabulario.
Validación	2.053	Vigilar el sobreajuste y decidir cuándo parar. No ajusta pesos.
Prueba	3.421	Medir una sola vez al final, como correos que el modelo nunca vio.

La división es **estratificada** (cada conjunto mantiene la proporción 62/38 de seguros y fraudes) y con `random_state=42` para que sea reproducible.

Tres defensas

- **Dropout(0.5)**: en cada paso de entrenamiento apaga al azar la mitad de los 64 valores que entrega la LSTM. La red no puede depender de un solo valor y aprende patrones más robustos. Al predecir, Dropout se desactiva y se usan todos.
- **Validación separada**: muestra cómo le va con correos que no usa para aprender.
- **Parada temprana**: si `val_loss` no mejora en 2 épocas, se detiene y recupera los pesos de la mejor época.

OJO

Si preguntan por qué la exactitud de entrenamiento es mayor que la de validación: es normal y esperable. Lo preocupante es que la validación empeore mientras el entrenamiento mejora. Eso es lo que vigila la parada temprana.

Métricas: leer los resultados

E2 E1

Todo sale de la **matriz de confusión** en los 3.421 correos de prueba:

	Predice seguro	Predice fraude
Es seguro (2.132)	**2.091** verdaderos negativos	**41** falsas alarmas
Es fraude (1.289)	**71** se escapan	**1.218** detectados

0,967 Exactitud (2.091 + 1.218) ÷ 3.421	**0,945** Recall de fraude 1.218 ÷ 1.289	**0,967** Precisión de fraude 1.218 ÷ (1.218 + 41)	**0,956** F1 de fraude promedio armónico

Qué significa cada una

- **Exactitud (accuracy)**: de todos los correos, cuántos acertó. Puede engañar: si 9 de cada 10 correos fueran seguros, un modelo que dice siempre "seguro" tendría 90%.
- **Recall (sensibilidad)**: de todos los fraudes reales, cuántos atrapó. Es la más importante, porque un fraude que pasa es el error caro.
- **Precisión**: de todo lo que marcó como fraude, cuánto lo era. Mide las falsas alarmas.
- **F1**: combina precisión y recall en un solo número.
- **AUC (0,991)**: la probabilidad de que, tomando un fraude y un correo seguro al azar, el modelo le dé más puntaje al fraude. 1 es perfecto, 0,5 es tirar una moneda. No depende del umbral.

OJO

En el ejemplo del profesor se imprime la accuracy con la etiqueta "Precisión del modelo". En español técnico, accuracy es **exactitud** y precision es **precisión**. Si lo preguntan, digan que son distintas y expliquen ambas.

El umbral

El modelo entrega una probabilidad; sobre 0,5 decimos "fraude". Si bajamos el umbral a 0,3, atrapamos más fraudes (sube el recall) pero bloqueamos más correos legítimos (baja la precisión). No hay umbral correcto: depende de qué error le duele más a la organización.

DILO ASÍ

"Para nosotros el falso negativo es más grave que la falsa alarma: un correo legítimo en cuarentena se recupera, una clave robada no. Por eso miramos el recall y, en la práctica, bajaríamos el umbral."

Las 4 pruebas con correos nuevos

Correo	Real	P(fraude)	Por qué
Banco: cuenta suspendida	Fraude	99,1%	urgencia, amenaza, verificar datos
Paquete: pagar $2,99	Fraude	97,8%	pago, plazo, actualizar datos
Soporte TI: confirmar cuenta	Fraude	4,1%	sin urgencia y con vocabulario académico, que en el dataset es casi siempre seguro
Coordinación: presentaciones	Seguro	0,4%	aviso normal

El error del Soporte TI es el resultado más útil para la defensa: muestra que el modelo detecta bien el fraude "clásico" pero no el bien redactado, y explica por qué la respuesta al problema es **parcial**.

¿Por qué empata con la regresión logística?

E1 E2

La comparación (paso 8) usa **TF-IDF + regresión logística**. TF-IDF da a cada palabra un peso según cuántas veces aparece en el correo y qué tan rara es en el resto. La regresión logística suma esos pesos y aplica una sigmoide: es, en el fondo, una sola neurona.

	LSTM	Regresión logística
Exactitud	0,967	0,967

Recall de fraude	0,945	0,940
Precisión de fraude	0,967	0,972
AUC	0,991	0,995

Empatan porque en este dataset las palabras sueltas ya separan casi todo: "verify", "suspended", "click" bastan. El orden casi no agrega información aquí. Eso no hace inútil a la LSTM: está pensada para fraudes donde el sentido depende del orden y del contexto, que son justamente los que hoy se escapan.

DILO ASÍ

"Lo medimos y lo decimos: en este dataset empatan. Elegimos la LSTM porque considera el orden de las palabras. Con datos más difíciles, el orden debería pesar más. Compararla con un modelo simple es lo que permite saber si el Deep Learning aporta."

OJO

No digan que la LSTM es mejor. Los números no lo muestran, y el profesor lo va a notar.

¿Y una CNN? (el curso vio CNN) -

Las CNN del curso son para imágenes: buscan patrones locales en una grilla de píxeles. Existe una versión 1D para texto que busca grupos de palabras vecinas, y funciona bien. La LSTM la elegimos porque lee el correo completo en orden y mantiene memoria a lo largo de él, que es lo que pide un texto como secuencia.

Riesgos, ética y mejoras

E1

- **Privacidad**: el modelo lee el contenido de correos que pueden tener datos personales (Ley 21.719 de protección de datos).
- **Incidentes**: la Ley 21.663, marco de ciberseguridad, obliga a las instituciones que regula a reportar incidentes a la ANCI.
- **Falsos negativos**: dan una falsa sensación de seguridad. El modelo apoya, no reemplaza la revisión humana ni la capacitación.
- **Falsas alarmas**: si son muchas, la gente deja de confiar en el filtro.

- **Datos que envejecen**: los atacantes cambian su forma de escribir, y ya usan IA para redactar mejor. Hay que reentrenar.
- **Sesgos del dataset**: inglés, Enron, listas académicas.

OJO

Otra limitación conocida: la LSTM no entiende bien la negación. En el ensayo, "We will never ask you to confirm your password" obtuvo 0,81 de fraude por las palabras "confirm" y "password".

Mejoras (el video debía cerrar con esto)

- Datos más variados, recientes y en español.
- Leer más de 200 palabras o el asunto y el remitente por separado.
- Bajar el umbral para subir el recall.
- Revisar señales sesgadas (vocabulario académico, enlaces mal detectados).
- Probar modelos preentrenados de lenguaje, que ya entienden contexto y negación.

Preguntas de defensa

Tapen la respuesta e intenten contestar en voz alta. E1: estudiante 1, E2: estudiante 2, aunque el profesor puede preguntarle cualquiera a cualquiera.

E1 1. ¿Qué problema resuelven, en una frase?

Clasificar automáticamente cada correo entrante como seguro o fraudulento a partir de su texto, con una probabilidad, para detener el phishing antes de que alguien caiga.

E1 2. ¿Por qué es importante resolverlo?

Basta que una persona caiga para comprometer una cuenta. Hay pérdidas de dinero, filtración de datos, horas de TI y obligaciones legales (Ley 21.663 y 21.719). Revisar a mano no escala.

E1 3. ¿Supervisado o no supervisado?

Supervisado: cada correo del dataset trae su etiqueta ("Safe" o "Phishing"). No supervisado sería sin etiquetas, como K-Means o Isolation Forest.

E1 4. ¿Por qué Deep Learning y no reglas fijas?

Las reglas se esquivan cambiando una palabra. Un modelo aprende patrones de miles de ejemplos y entrega una probabilidad.

E1 5. ¿Por qué una LSTM y no una red densa?

Un correo es una secuencia y el orden importa. La red densa recibe todo junto y no ve el orden; la LSTM lee palabra por palabra y guarda memoria.

E1 6. ¿Qué diferencia a una LSTM de una RNN simple?

La memoria de celda y las tres compuertas. La RNN simple sufre desvanecimiento del gradiente y olvida lo que leyó hace muchas palabras.

E1 7. Explique las compuertas de la LSTM.

Olvido: qué borrar de la memoria. Entrada: qué guardar de la palabra nueva. Salida: qué parte de la memoria entregar. Cada una es una sigmoide entre 0 y 1.

E1 8. ¿Qué es un peso y qué es un sesgo?

El peso multiplica cada entrada y dice cuánto importa. El sesgo se suma al final y corre el punto de partida. Ambos se aprenden.

E1 **9. ¿Por qué sigmoide en la salida?**

Aplasta cualquier número entre 0 y 1, que se interpreta como probabilidad de fraude. Es lo natural para dos clases.

E1 **10. ¿Qué función de pérdida usan y qué mide?**

Entropía cruzada binaria. Mide qué tan lejos está la probabilidad de la respuesta real y castiga mucho equivocarse con seguridad. Es la pareja natural de la sigmoide.

E1 **11. ¿Qué es la retropropagación?**

El cálculo de cuánto aportó cada peso al error, desde la salida hacia atrás, capa por capa. Entrega los gradientes que usa el optimizador.

E1 **12. ¿Qué es el descenso del gradiente? ¿Y Adam?**

Mover cada peso un poco en la dirección que baja el error. Adam es una versión que ajusta sola el tamaño del paso de cada peso y usa la dirección de los pasos anteriores.

E1 **13. ¿Qué es una época y qué es un lote?**

Época: una pasada por todos los correos de entrenamiento. Lote: grupo de 32 correos tras el cual se actualizan los pesos. Son 364 actualizaciones por época.

E1 **14. ¿Qué es el sobreajuste y cómo lo evitaron?**

Memorizar el entrenamiento y fallar con datos nuevos. Lo controlamos con Dropout 0,5, validación separada y parada temprana con paciencia 2.

E1 **15. ¿Qué riesgos éticos tiene?**

Lee contenido privado (Ley 21.719), puede bloquear correos legítimos, puede dar falsa seguridad y no reemplaza la revisión humana.

E1 **16. ¿Qué es más grave: falso positivo o falso negativo?**

El falso negativo: un fraude que llega a la bandeja. Un correo legítimo en cuarentena se recupera; una clave robada no.

E2 **17. ¿Por qué limpian el texto?**

Para quitar ruido (HTML, mayúsculas, símbolos) y atajos que no generalizan, como la palabra "enron". Así el modelo aprende del contenido.

E2 **18. ¿Qué tiene que ver Enron?**

Muchos correos seguros vienen de los correos públicos de Enron: "enron" está en el 20,3% de los seguros y 0% de los fraudes. Se borra para que el modelo no aprenda "si dice enron, es seguro".

E2 **19. ¿Qué hace TextVectorization?**

Arma un vocabulario con las 20.000 palabras más frecuentes del entrenamiento y convierte cada correo en 200 números. Rellena con ceros o corta.

E2 **20. ¿Por qué adapt() solo con entrenamiento?**

Para no filtrar información de la prueba al modelo (fuga de datos). La prueba debe simular correos nunca vistos.

E2 **21. ¿Qué es el Embedding?**

Una tabla de 20.000 × 64 que convierte cada palabra en un vector de 64 valores. Se aprende al entrenar y deja cerca las palabras que se usan parecido.

E2 **22. ¿Para qué sirve mask_zero=True?**

Avisa a la LSTM que los ceros son relleno y que no los lea.

E2 **23. ¿Por qué 200 palabras?**

Cubre más que la mediana (160) y mantiene el entrenamiento rápido. Lo que viene después de la palabra 200 se pierde: es una limitación.

E2 **24. ¿Por qué separan validación y prueba?**

La validación se mira durante el entrenamiento para decidir cuándo parar; si usáramos la prueba para eso, el resultado final estaría inflado. La prueba se usa una vez.

E2 **25. ¿Qué significa stratify y random_state=42?**

Stratify mantiene la proporción de seguros y fraudes en cada conjunto. random_state fija el azar para que la división sea siempre la misma.

E2 **26. ¿Qué hace EarlyStopping?**

Vigila val_loss. Si no mejora en 2 épocas, detiene el entrenamiento y recupera los pesos de la mejor época.

E2 **27. ¿Cuántos parámetros tiene y de dónde salen?**

1.313.089. Embedding 20.000×64 = 1.280.000; LSTM 4×(64×128+64) = 33.024; Dense 64+1 = 65.

E2 **28. Interprete la matriz de confusión.**

De 1.289 fraudes detectó 1.218 y se escaparon 71. De 2.132 seguros, 41 fueron falsas alarmas.

E2 **29. ¿Diferencia entre exactitud, precisión y recall?**

Exactitud: aciertos sobre el total. Precisión: de lo marcado como fraude, cuánto lo era. Recall: de los fraudes reales, cuántos detectó.

E2 **30. ¿Qué es el AUC?**

Qué tan bien separa ambas clases con cualquier umbral: la probabilidad de que un fraude reciba más puntaje que un correo seguro. 1 perfecto, 0,5 azar. El nuestro: 0,991.

E2 **31. Si la regresión logística empata, ¿para qué la LSTM?**

Lo decimos con honestidad: en este dataset las palabras sueltas bastan. La LSTM considera el orden, que debería importar con fraudes mejor escritos. Comparar es lo que permite saberlo.

E2 **32. ¿Por qué falló el correo de Soporte TI?**

Está bien redactado, sin urgencia ni amenazas, y usa vocabulario académico que en el dataset aparece casi solo en correos seguros (university: 14,7% vs 0,9%).

E2 **33. ¿Qué cambiaría si bajan el umbral?**

Detecta más fraudes (sube el recall) a cambio de más falsas alarmas (baja la precisión). Es una decisión de la organización.

E2 **34. ¿La solución resuelve el problema?**

Parcialmente. Detecta el 94,5% de los fraudes con pocas falsas alarmas, pero se escapan los fraudes bien redactados.

E2 **35. ¿Qué mejorarían?**

Datos más variados y en español, leer más de 200 palabras, ajustar el umbral, revisar señales sesgadas y probar modelos de lenguaje preentrenados.

E2 **36. ¿Usaron GPU? ¿Importa?**

Usamos Colab con su entorno. La GPU solo acelera el entrenamiento; no cambia el resultado. En el ensayo en CPU entrenó en poco más de un minuto.

Cifras del notebook `deteccion_phishing_lstm.ipynb` del repositorio. Si en el video salieron otros números, usen los del video en la defensa.